

Parametric Curve Parameter Recovery

This solves the assignment problem: find θ , M , and X for this curve

$$\begin{aligned}x(t) &= t \cos(\theta) - e^{(M|t|)} \sin(0.3t) \sin(\theta) + X \\y(t) &= 42 + t \sin(\theta) + e^{(M|t|)} \sin(0.3t) \cos(\theta)\end{aligned}\quad \text{for } 6 < t < 60$$

using only the points in `xy_data.csv`. We are not told which point matches which t value.

Answer

```
theta = 29.999973 deg    (0.5235983 rad)
M      = 0.030000
X      = 54.999998
```

Desmos submission string:

```
\left(t*\cos(0.5235983)-e^{\{0.0300000\left|t\right|\}}\cdot\sin(0.3t)
\sin(0.5235983)+55.0000,42+t*\sin(0.5235983)+e^{\{0.0300000\left|t\right|\}}
\cdot\sin(0.3t)\cos(0.5235983)\right)
```

How I solved it

Step 1: Examining the structure of the equations

Rearranging the two equations shows that they describe a simpler curve $(t, v(t))$, where $v(t) = e^{(M|t|)} \sin(0.3t)$, rotated by θ and then translated by $(X, 42)$:

$$\begin{aligned}x - X &= t \cos(\theta) - v(t) \sin(\theta) \\y - 42 &= t \sin(\theta) + v(t) \cos(\theta)\end{aligned}$$

In other words: the curve $(t, v(t))$ is rotated by θ , then shifted by $(X, 42)$. That is the full structure of the equations.

Please see [maths.jpeg](#) for the derivation

Step 2: The problem — t is unknown for every point

Normally, fitting a curve to a set of points like this would require treating the t value of every single point as an unknown too (1500 of them), in addition to θ , M , and X . This adds a large number of extra unknowns and makes the problem considerably harder.

However, since the transformation is a rotation, it can be undone. Given a candidate θ and X , rotating a data point backwards gives an x -value that is exactly what t must be for that point:

$$\begin{aligned}t_i' &= (x_i - X) \cos(\theta) + (y_i - 42) \sin(\theta) \\v_i' &= -(x_i - X) \sin(\theta) + (y_i - 42) \cos(\theta)\end{aligned}$$

This means t does not need to be guessed at all — it is recovered directly once θ and X are chosen. As a result, the problem reduces to searching over only 3 unknowns: θ , M , X .

Step 3: Turning this into something we can optimize

For each point, we now have a t_i' and v_i' from the rotation above. If our guess for θ , M , X is correct, then v_i' should match what the model formula predicts at that same t_i' . So we check the difference between the two:

```
residual_i = v_i' - e^(M*|t_i'|) * sin(0.3 * t_i')
```

We add up the square of this difference for all 1500 points, and try to make that total as small as possible. This total only becomes zero for every point at once when θ , M , X are the correct values.

Step 4: Finding θ , M , X

I used two steps here, in this order, for a reason.

$\sin(0.3t)$ goes up and down many times as t moves from 6 to 60. That means the total error (the thing we are trying to make small) does not just have one smooth dip — it has several dips, and only one of them is the real lowest point. If we started with a method that only looks at which direction is downhill from a single starting guess, it could easily slide into the wrong dip and get stuck there, thinking it found the best answer when it actually did not.

So I used two different tools, one for each job:

1. **scipy.optimize.differential_evolution** first — this searches the whole allowed range (θ 0-50 deg, M -0.05 to 0.05, X 0-100) by keeping many guesses at once and mixing them together over many rounds, instead of following one path downhill. Because it looks at many spots across the whole range at the same time, it is much less likely to get stuck in the wrong dip. Its job is just to get us somewhere close to the correct answer, not to be exact.
2. **Least squares** (`scipy.optimize.least_squares`) second — starting from what differential evolution found. Once we are already close to the correct dip, least squares is very good and fast at walking the rest of the way down to the exact lowest point. `least_squares` requires a "good enough" initial guess to work well and we got that "good enough" from differential evolution.

I did not use least squares by itself from the start, because it could have locked onto the wrong dip. I did not use differential evolution by itself either, because it is good at finding the right area but not at getting a precise, exact answer. Using both together, in this order, gets the best of both.

Step 5: Checking the answer is correct

- I put the rounded values (30 deg, 0.03, 55) back into the original equations, and they reproduce the given data almost exactly ($\sim 1e-5$ error). This is a strong sign these are the real values, not just a close guess.
- I also plotted the curve with these values on top of the data points (`fit_comparison.png`), and they match closely.

Other methods I thought about, and why I did not use them

- **Grid search** over θ , M , X would also work, but it needs a very fine grid to get this level of accuracy, which makes it much slower for no extra benefit.
- **Training a neural network** to guess θ , M , X from the points could work too, but we already know the exact formula here. Training a model would take much more work (making training data, training the model) and would likely be less accurate than just fitting the 3 numbers directly.

References

- Differential Evolution algorithm — watched this to understand how the method works before using `differential_evolution`: <https://youtu.be/xwR7WbKtylg>
- Nonlinear least squares — watched this to understand how the method works before using `least_squares`: <https://youtu.be/8evmj2L-iCY>
- `scipy.optimize.differential_evolution` docs — used to understand how to use the function: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html
- `scipy.optimize.least_squares` docs — used to understand how to use the function: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html

Files

- `fit_parameters.py` — loads `xy_data.csv`, runs the fit, and prints the parameters.
- `plot_fit.py` — draws the fitted curve on top of the data points, and saves `fit_comparison.png`.
- `xy_data.csv` — the given data.
- `fit_comparison.png` — the picture used to check the fit visually.
- `requirements.txt` — the Python packages needed to run the code.
- `maths.jpeg` - shows the derivation

How to run it

```
pip install -r requirements.txt
python fit_parameters.py --csv xy_data.csv
python plot_fit.py --csv xy_data.csv --out fit_comparison.png
```